Apuntes de Programación (Java)

Oct 20, 2026 · @Alvaro

1. Tipos de datos

Los tipos primitivos se escriben en minúscula; `String` va en mayúscula porque es una clase.

Categoría	Tipo	Rango / valores	Ejemplo
Entero	`byte`	-128 a 127	`byte edad = 25;`
Entero	`short`	-32.768 a 32.767	`short año = 2026;`
Entero	`int`	-2.147.483.648 a 2.147.483.647	`int total = 150000;`
Entero	`long`	-9.223.372.036.854.775.808 a 9.223.372.036.854.775.807	`long dni = 12345678L;`
Decimal	`float`	Precisión simple (~7 dígitos)	`float precio = 9.99f;`
Decimal	`double`	Precisión doble (~15 dígitos)	`double media = 7.85;`
Carácter	`char`	Un solo carácter, comillas simples	`char letra = 'A';`
Booleano	`boolean`	`true` o `false`	`boolean aprobado = true;`
Texto	`String`	Cadena de caracteres, comillas dobles	`String nombre = "Ana";`

- Por defecto usa `int` para enteros y `double` para decimales.
- `long` lleva `L` al final y `float` lleva `f`.

2. A tener en cuenta

- **Clases:** primera letra en mayúscula (`Alumno` , `CalculadoraNotas`).
- **Variables y métodos:** nombres representativos en camelCase (`notaMedia` , no `x`).
- **`break` :** solo dentro de `switch` ; no lo uses para salir de bucles.
- **Estructura:** código bien indentado y ordenado, un bloque por tarea.

- **Enunciado:** usa exactamente los nombres de métodos, variables y ficheros que pide el ejercicio.
- **Buffer del Scanner:** después de `nextInt()` o `nextDouble()` queda un salto de línea pendiente. Antes de leer un `String` con `nextLine()`, límpialo:

```
int edad = sc.nextInt();
sc.nextLine();              // limpia el buffer
String nombre = sc.nextLine();
```

3. Depurar (debug)

Depurar es ejecutar el programa paso a paso para ver qué valor tiene cada variable y encontrar dónde falla.

1. **Pon breakpoints** (clic en el margen junto al número de línea) donde sospechas el error: antes de un `if`, dentro de un bucle o tras leer datos.
2. **Ejecuta en modo Debug**, no en modo normal. El programa se para en el primer breakpoint.
3. **Avanza paso a paso:**
   - *Step Over*: ejecuta la línea actual sin entrar en los métodos.
   - *Step Into*: entra dentro del método que se llama en esa línea.
   - *Resume*: continúa hasta el siguiente breakpoint.
4. **Revisa las variables** en el panel de variables y comprueba si tienen el valor esperado.
5. **Corrige y repite** hasta que el resultado sea el correcto.

Consejo: no llenes el código de breakpoints; ponlos solo donde aporten información.